

whitenoise — User Guide

Stochastic White Noise Analysis with Memory (SWNA)

Package version 0.3.0 · Python ≥3.9 · May 16, 2026

1. Overview

whitenoise is a research package implementing Stochastic White Noise Analysis (SWNA) with Memory. The framework models a fluctuating observable as a stochastic integral driven by white noise, where a memory kernel encodes how past fluctuations influence future dynamics.

The central quantity is the *Mean Square Displacement* (MSD), which characterises how the spread of a fluctuating observable grows with lag Δ :

$$\text{MSD}(\Delta) = (1 / (N - \Delta)) \cdot \sum_{i=0}^{N-\Delta-1} [x(i+\Delta) - x(i)]^2$$

Fitting a theoretical SWNA model to the empirical MSD yields the **memory parameter** μ , which quantifies the strength and direction of memory in the system.

The package always reports μ (and its 95% confidence interval) directly. Where a `result.regime` classification is available, it is **model-family specific** — two independently sourced schemes apply to two different model families, and they are never merged into a single generic μ -only table:

- **Cosine / sine models** admit a short-time ($T \ll 1$) reduction of the MSD to a power law, $\text{MSD} \approx c \cdot T^\alpha$, with $\alpha = 2\mu - 1$. Classification is on α , not μ directly: $\alpha < 1$ subdiffusive, $\alpha = 1$ Brownian, $1 < \alpha < 2$ superdiffusive, $\alpha > 2$ hyperballistic. Source: Elnar et al. (2021, *Climate Dynamics*).
- **Exponential model** does not reduce to a clean power law over the fitting range, so the α -based labels above do not apply. Instead classification comes from the power-law scaling exponent $(\mu-1)/2$ in the memory function $f(t-\tau)h(\tau) = (t-\tau)^{(\mu-1)/2} \cdot e^{-\beta/2\tau}/\tau$:
 - $\mu = 1$: exponent $(\mu-1)/2 = 0$, so the power-law term disappears and $f(t-\tau)h(\tau) = e^{-\beta/2\tau}/\tau$ — fluctuation behavior is dominated purely by the exponential damping term: a **memoryless** process.
 - $\mu > 1$: exponent $(\mu-1)/2 > 0$, so $(t-\tau)^{(\mu-1)/2}$ increases as $t-\tau$ increases — the memory effect of an earlier fluctuation at $\tau \ll t$ is stronger. **Non-Markovian, long memory.**
 - $\mu < 1$: exponent $(\mu-1)/2 < 0$, so $(t-\tau)^{(\mu-1)/2}$ decreases as $t-\tau$ increases — the memory effect of an earlier fluctuation at $\tau \ll t$ is weaker (though it strengthens again as τ approaches t). **Non-Markovian, short memory.**

Source: Tey et al. (2024, *Physica Scripta* 100, 015243).

- **All other models** (fBm and the stub models) have no automatic classification (`result.regime` is `None`) until their own reduction/source is confirmed. $\mu = 1$ (or $H = 0.5$ for fBm) is always ordinary Brownian motion as a plain mathematical fact, independent of any classification scheme.

Scope: The package accepts any sequential 1D observable — not just time series. Column 1 is the independent variable (time, index, frequency, distance, occurrence number, etc.); column 2 is the observable. The term “time” is used loosely throughout this guide.

2. Installation

```
pip install -e .          # development install (editable, from the repo root)
pip install whitenoise-swna # once published to PyPI
```

Required dependencies: numpy ≥ 1.20 , scipy ≥ 1.7 , matplotlib ≥ 3.4 , pandas ≥ 1.3 .

Optional: openpyxl ≥ 3.0 for Excel export (`pip install openpyxl`).

```
import whitenoise as wn    # standard import alias used throughout this guide

print(wn.__version__)      # '0.1.0'
```

3. Quickstart

```
import whitenoise as wn

# --- Analyze a single CSV ---
result = wn.analyze('sunspot.csv', model='exponential')
result.summary()          # prints formatted parameter table
wn.plot_diagnostics(result) # four-panel figure: series, MSD, PDF, params

# --- Inspect data before fitting ---
time, values, meta = wn.read_csv('sunspot.csv')
wn.plot_series(time, values, metadata=meta)    # raw series
lags, msd = wn.compute_msd(values)
wn.plot_msd_empirical(lags, msd, metadata=meta) # MSD shape check

# --- Optional detrending before analysis ---
fluct = wn.detrend(values, method='moving_average', window=11)
result = wn.analyze(fluct, model='cosine', label='Sunspot fluctuations')

# --- Compare multiple datasets ---
comparison = wn.compare({
    'Sunspots': 'sunspot.csv',
    'Earthquakes': 'earthquake_ph.csv',
}, model='exponential')
print(comparison.summary_df)
wn.publish_comparison(comparison)

# --- Batch process a folder ---
batch = wn.batch_analyze('data/xray_binaries/', model='cosine')
wn.export_summary(batch, 'results.csv')
```

```
# --- Try all models on one file ---
search = wn.batch_model_search('sunspot.csv')
print(search.summary_df[['model', 'r_squared']].to_string())
```

4. CSV Input Format

The package reads **2-column CSV files** (or multi-column for batch mode) with a required header row. The header encodes column names and physical units in the format `name [unit]`:

```
time [months], sunspot_number [count]
1, 12.4
2, 15.1
3, 14.8
```

Rule	Detail
Column 1	Independent variable (time, index, occurrence number, distance, ...). Always.
Column 2	Observable to analyse. Always.
Header format	<code>name [unit]</code> — space before bracket is optional: <code>co2[ppm]</code> also works.
Unitless column	Use empty brackets: <code>flux []</code> or <code>value [unitless]</code> .
No brackets	Also accepted: <code>time</code> , <code>distance</code> — unit defaults to empty string.
Case	Column names are case-insensitive internally; preserved as written for display.
Multi-column	<code>time [yr]</code> , <code>co2 [ppm]</code> , <code>temperature [°C]</code> — use <code>read_csv_multi()</code> or <code>batch_analyze()</code> .

Unit-based axis labels

Units are used exclusively for automatic axis labelling. No manual labelling is ever needed.

Header	Axis label produced
<code>sunspot_number [count]</code>	<code>sunspot_number (count)</code>
<code>flux []</code>	<code>flux</code>
<code>value [unitless]</code>	<code>value</code>
<code>time [months]</code>	<code>time (months)</code>

Automatic validation warnings (non-fatal)

On read, the package runs **3** checks and prints a  warning if triggered. Execution always continues — none of these stop the pipeline; there is no check for non-monotonic time values.

Code	Condition	Warning (paraphrased)
------	-----------	-----------------------

V1	Column 1 spread is unusually large relative to its mean (<code>std > 100 × mean </code>) — possible column swap.	⚠ Column order warning: column 1 ('{x_name}') has unusually large spread...
V2	Column 1 name is not a commonly recognized time/index name (e.g. not <code>time</code> , <code>t</code> , <code>index</code> , <code>step</code> , <code>lag</code> , <code>year</code> , <code>date</code> , ...). This is informational only — column 1 doesn't have to be time (it can be distance, frequency, voltage, etc.), so this warning is expected and safe to ignore for non-time data.	⚠ Column 1 name warning: '{x_name}' is not a commonly recognized time or index name...
V3	Column 2's unit implies non-negative values (e.g. <code>count</code> , <code>flux</code> , <code>ppm</code> , <code>probability</code> , ...) but negative values were found.	⚠ Value range warning: column '{y_name}' has unit '{y_unit}' which typically implies non-negative values, but N negative value(s) were found...

read_csv()

```
read_csv(path: str) -> tuple[np.ndarray, np.ndarray, dict]
```

Read a 2-column whitenoise-format CSV file.

Parameter	Type	Default	Description
<code>path</code>	<code>str</code>	<i>required</i>	Path to the CSV file.

Returns: a 3-tuple (`time`, `values`, `metadata`)

- `time` — ndarray shape (N,), column 1 values.
- `values` — ndarray shape (N,), column 2 values.
- `metadata` — dict with keys: `x_name`, `x_unit`, `x_label`, `y_name`, `y_unit`, `y_label`, `source_file`, `n_points`.

Raises: `FileNotFoundError` with message `X File not found: {path}`; `ValueError` for malformed format.

```
time, values, meta = wn.read_csv('sunspot.csv')
print(meta['y_label']) # 'sunspot_number (count)'
print(meta['x_label']) # 'time (months)'
print(meta['n_points']) # e.g. 300
print(meta['y_name']) # 'sunspot_number'
print(meta['y_unit']) # 'count'
```

read_csv_multi()

```
read_csv_multi(path: str) -> list[tuple[np.ndarray, np.ndarray, dict]]
```

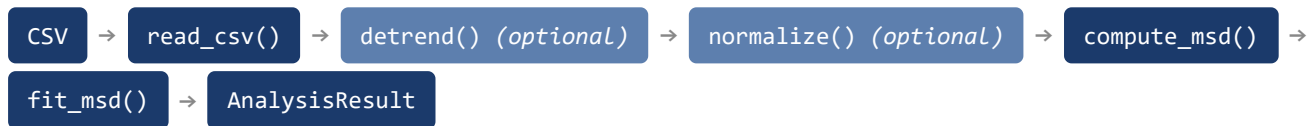
Read a multi-column CSV. Returns one (`time`, `values`, `metadata`) tuple per non-time column. Used internally by `batch_analyze()` in multi-column mode.

```

results = wn.read_csv_multi('climate.csv')
# CSV has: time [yr], co2 [ppm], temperature [°C]
len(results)                # 2 (co2 and temperature)
results[0][2]['y_name']     # 'co2'
results[1][2]['y_name']     # 'temperature'

```

5. The Full Pipeline



The `analyze()` function runs the entire pipeline in one call. You can also call each step individually for full control.

Step	Function	Purpose	Mandatory?
1. Load	<code>read_csv(path)</code>	Read CSV, parse headers, validate	Yes (or array input)
2. Detrend	<code>detrend(values, method=...)</code>	Remove trend to extract fluctuations	No (researcher decides)
3. Normalize	<code>normalize(values, method=...)</code>	Scale data (rarely needed; N param absorbs amplitude)	No
4. MSD	<code>compute_msd(values)</code>	Compute empirical MSD array	Yes
5. Fit	<code>fit_msd(lags, msd, model=...)</code>	Fit theoretical model, extract μ with 95% CIs	Yes
6. Result	<code>AnalysisResult</code>	Container: params, MSD, plots, export	Yes

Note on normalization: Pre-analysis normalization is *not standard* in the published SWNA work. The normalization scalar N in the fit ($N \cdot \text{msd_model}$) absorbs data amplitude and scale automatically — the memory parameter μ is unaffected by amplitude. Normalization is only useful in rare edge cases of extreme numerical scale differences.

6. analyze() and AnalysisResult

analyze()

```

analyze(source, model='cosine', label='', time=None, detrend_method=None,
        normalize=False, max_lag_fraction=1.0, fit_kwargs=None, verbose=True)
-> AnalysisResult

```

Run the full SWNA pipeline on a CSV file or a data array.

Parameter	Type	Default	Description
-----------	------	---------	-------------

source	str or array-like	<i>required</i>	CSV file path (str) or 1D data array. When str, labels and units are read automatically from the header.
model	str	'cosine'	SWNA model name. Run <code>wn.list_models()</code> for options.
label	str	''	Dataset name. Auto-set from CSV <code>y_name</code> when source is a CSV. Required when source is an array.
time	array-like or None	None	Time axis. Only used when source is an array; ignored for CSV input (time comes from the file).
detrend_method	str or None	None	If not None, calls <code>detrend(values, method=detrend_method)</code> before MSD. Choices: 'linear', 'polynomial', 'mean', 'moving_average'.
normalize	bool	False	If True, z-score normalizes values after detrending and before MSD.
max_lag_fraction	float	1.0	Fraction of lags [0, 1] to use in fitting. Default 1.0 uses all $N//2$ lags.
fit_kwargs	dict or None	None	Extra keyword arguments forwarded to <code>fit_msd()</code> , e.g. <code>{'p0': [1.5, 0.01], 'bounds': ...}</code> .
verbose	bool	True	If True, print ✓ progress lines and final R^2 summary. Fitting quality warnings are always printed regardless.

Returns: `AnalysisResult`

Raises: `ValueError` if model name is not in the MODELS registry (message lists valid names);
`ValueError` if source is an array but label is not provided.

```
# From CSV (recommended for research)
result = wn.analyze('sunspot.csv', model='exponential')
result.summary()

# From array (after manual detrending)
fluct = wn.detrend(values, method='moving_average', window=11)
result = wn.analyze(fluct, model='cosine', label='Sunspot fluctuations')

# Suppress all progress output
result = wn.analyze('data.csv', model='cosine', verbose=False)

# Pass custom initial guess to fitting
```

```
result = wn.analyze('data.csv', model='cosine',
                    fit_kwargs={'p0': [2.0, 0.005]})
```

Tip: When `analyze()` is called with a CSV path, the `dataset_name` field of the result is set to the filename stem (e.g. `'sunspot'` for `sunspot.csv`). The `label` property (used in plot titles) is set to the CSV `y_name` unless you override it.

AnalysisResult fields

Field	Type	Description
<code>dataset_name</code>	str	CSV filename stem, or the label you supplied for array input.
<code>model</code>	str	SWNA model name used for fitting.
<code>fit</code>	FitResult or None	Full fitting results. None if fitting failed.
<code>lags</code>	ndarray	Lag array from <code>compute_msd()</code> , integers $1 \dots N//2$.
<code>msd_empirical</code>	ndarray	Empirical MSD values at each lag.
<code>values</code>	ndarray	Preprocessed observable (after detrend/normalize if applied).
<code>time</code>	ndarray	Independent variable array (from CSV col 1, or supplied <code>time=</code>).
<code>metadata</code>	dict	Keys: <code>x_name</code> , <code>x_unit</code> , <code>x_label</code> , <code>y_name</code> , <code>y_unit</code> , <code>y_label</code> , <code>source_file</code> , <code>n_points</code> .

AnalysisResult properties

Classification (`.alpha`, `.memory_exponent`, `.regime`) is **model-family specific** — only one of the first two is ever non-`None` for a given result, and `.regime` itself is `None` for models with no confirmed classification scheme. See Section 1 for the full rationale and citations.

Property	Type	Description
<code>.alpha</code>	float or None	$\alpha = 2\mu - 1$, the short-time power-law exponent. Defined only for <code>cosine</code> / <code>sine</code> (Elnar et al. 2021); <code>None</code> for all other models.
<code>.memory_exponent</code>	float or None	$(\mu - 1)/2$, the power-law scaling exponent in the exponential model's memory function. Defined only for <code>exponential</code> (Tey et al. 2024); <code>None</code> for all other models.
<code>.regime</code>	str or None	Classification label dispatched by model: <code>'subdiffusive'</code> / <code>'brownian'</code> / <code>'superdiffusive'</code> / <code>'hyperballistic'</code> for cosine/sine (via <code>.alpha</code>); <code>'memoryless'</code> / <code>'non-Markovian, long</code>

```
memory' / 'non-Markovian, short memory' for exponential (via
.memory_exponent ); None for fbm and all stub models.
```

No `.mu` shortcut. There is no convenience property for the primary memory parameter — read it from `result.fit.params['mu']` (or `result.fit.params['H']` for fbm, `result.fit.params['a']` for `exp_plateau`). See the `FitResult` table in Section 8.

AnalysisResult methods

.summary() — Prints a formatted analysis summary block to stdout (returns `None`). Real example, exponential model:

```
=====
SWNA Analysis Summary
=====

Dataset   : sunspot_data
Model     : exponential
Points    : 300
Lags used : 150

Parameters:
  μ       = 1.1500 ± 0.0082
  β       = 0.1000 ± 0.0030
  N       = 2.4312 ± 0.0441
R² (pure MSD) = 0.6924
R² (N·MSD)   = 0.9195 ← selected
(μ-1)/2     = 0.0750
Memory      : non-Markovian, long memory (Tey et al. 2024)

Units      : x=time (months), y=sunspot_number (count)
=====
```

Always prints R^2 for **both** fit modes (pure MSD, N·MSD) with the selected mode flagged — see the dual-fit note in Section 8. The α /regime or $(\mu-1)/2$ /memory lines only appear when the fitted model has a classification scheme (cosine/sine or exponential); they are omitted entirely for fbm and stub models.

No export/plot shortcuts on AnalysisResult. `AnalysisResult` has no `.to_dict()`, `.to_csv()`, `.plot_all()`, `.plot_msd()`, or `.plot_pdf()` methods — `.summary()` is the only method. To export or plot a result, call the standalone functions instead:

```
wn.export_csv(result, 'my_result.csv') # see Section 12
wn.plot_msd(result)                   # see Section 10
wn.plot_pdf(result)
wn.plot_diagnostics(result)           # 4-panel: series, MSD, PDF, params
```

7. Preprocessing (Optional)

These helpers are *optional*. Call them manually before `analyze()` if your data has a non-stationary trend or needs smoothing. The pipeline never calls them automatically unless you pass `detrend_method=` to `analyze()`.

When to detrend: The SWNA framework assumes the observable represents *fluctuations* around some baseline. If your data has a clear long-term trend (e.g. a rising CO₂; baseline, a linear drift), remove it first. If the data is already fluctuating around zero, no detrending is needed.

detrend()

```
detrend(values, method='linear', poly_order=1, window=7) -> np.ndarray
```

Remove a trend from a time series to extract fluctuations.

Parameter	Type	Default	Description
<code>values</code>	array-like (1D)	<i>required</i>	Input time series.
<code>method</code>	str	<code>'linear'</code>	Detrending method (see table below).
<code>poly_order</code>	int	<code>1</code>	Polynomial degree. Used only when <code>method='polynomial'</code> . Ignored otherwise.
<code>window</code>	int	<code>7</code>	Moving average window size. Used only when <code>method='moving_average'</code> . Must be odd; even values are silently incremented by 1.

method=	What it does	When to use
<code>'linear'</code>	Fits a degree-1 polynomial and subtracts it.	Long-term linear drift (e.g. population growth).
<code>'polynomial'</code>	Fits a degree- <code>poly_order</code> polynomial and subtracts it.	Smooth nonlinear trend (e.g. CO ₂ ; seasonal + secular).
<code>'mean'</code>	Subtracts the global mean only.	Data already stationary; just removes DC offset.
<code>'moving_average'</code>	Subtracts a uniform moving average of width <code>window</code> .	Slowly varying baseline (e.g. sunspot cycle).

Returns: ndarray same length as input (fluctuations around removed trend).

Raises: `ValueError` with `X` prefix for unknown method.

```
# Sunspot detrending – remove slow baseline cycle
time, values, meta = wn.read_csv('sunspot.csv')
fluct = wn.detrend(values, method='moving_average', window=11)
result = wn.analyze(fluct, model='exponential', label='Sunspot fluctuations')
```

```
# CO2 polynomial detrend
time, co2, meta = wn.read_csv('co2.csv')
fluct = wn.detrend(co2, method='polynomial', poly_order=2)
result = wn.analyze(fluct, model='cosine', label='CO2 fluctuations')
```

There are also two advanced detrending functions, both exported at `wn.*` like `detrend()` but **not** reachable via `analyze(detrend_method=...)` — call them manually before `analyze()`, same as `detrend()` itself.

detrend_psd()

```
detrend_psd(values, threshold=0.5) -> np.ndarray
```

Removes dominant periodic components via Power Spectral Density analysis: (1) FFT the signal, (2) normalize the PSD so the dominant peak = 1.0, (3) zero out frequency bins whose normalized power exceeds `threshold`, (4) inverse FFT. Useful for data with strong deterministic periodic cycles (e.g. predator–prey population oscillations).

Parameter	Type	Default	Description
<code>values</code>	array-like (1D)	<i>required</i>	Input time series.
<code>threshold</code>	float	<code>0.5</code>	Normalized PSD cutoff (0–1). Components above this are removed. Lower = more aggressive filtering. Typical choices: 0.2, 0.3, 0.5, 1.0.

Returns: ndarray same length as input.

```
# Algae predator-prey data – remove dominant cycle
fluct = wn.detrend_psd(algae_values, threshold=0.3)
result = wn.analyze(fluct, model='cosine', label='Algae fluctuations')
```

detrend_fingerprint()

```
detrend_fingerprint(values, cycle_length=None, trough_indices=None) -> np.ndarray
```

Cycle averaging (“fingerprinting”): (1) segment the signal at cycle troughs, (2) align all segments at their maxima by zero-padding the start, (3) equalize segment lengths by zero-padding the end, (4) compute the fingerprint as the mean of all padded segments, (5) subtract the fingerprint from each original segment (padding excluded). Useful when cycles have similar shapes but vary in timing/period.

Parameter	Type	Default	Description
<code>values</code>	array-like (1D)	<i>required</i>	Input time series with a dominant repeating cycle.

<code>cycle_length</code>	int or None	None	If given, splits the signal into equal-length chunks of this size. Simplest option when cycles are uniform.
<code>trough_indices</code>	list[int] or None	None	Explicit cycle-boundary indices. If neither this nor <code>cycle_length</code> is given, troughs are auto-detected via <code>scipy.signal.find_peaks</code> .

Returns: ndarray same length as input.

Raises: `ValueError` with `X` prefix if no valid cycles can be identified.

```
# Algae predator-prey – repeating predator-prey cycle
fluct = wn.detrend_fingerprint(algae_values, cycle_length=20)
result = wn.analyze(fluct, model='cosine', label='Algae residuals')
```

normalize()

```
normalize(values, method='zscore') -> np.ndarray
```

Parameter	Type	Default	Description
<code>values</code>	array-like (1D)	<i>required</i>	Input time series.
<code>method</code>	str	'zscore'	Normalization method (see table below).

method=	Formula	Result range	Notes
'zscore'	$(x - \text{mean}) / \text{std}$	mean=0, std=1	Standard z-score.
'minmax'	$(x - \text{min}) / (\text{max} - \text{min})$	[0, 1]	Shifts and scales to unit interval.
'mean'	$x / \text{mean}(x)$	mean=1	Preserves shape; mean normalized to 1.
'maxabs'	$x / \text{max}(x)$	[-1, 1]	Preserves zero and sign. Best for oscillating or detrended data where zero has physical meaning.

Raises: `ValueError` with `X` prefix for unknown method; also for `'maxabs'` when all values are zero.

smooth()

```
smooth(values, window=5, method='moving_average') -> np.ndarray
```

Smooth a time series. Output is always the same length as input.

Parameter	Type	Default	Description
-----------	------	---------	-------------

values	array-like (1D)	<i>required</i>	Input time series.
window	int	5	Kernel width (moving_average) or sigma=window/4 (gaussian). Odd recommended; even silently incremented.
method	str	'moving_average'	'moving_average' (uniform box filter) or 'gaussian' (Gaussian kernel via scipy.ndimage).

Returns: ndarray same length as input.

Raises: `ValueError` with `X` prefix for unknown method. Prints `Δ` if even window given.

8. Core Functions

compute_msd()

```
compute_msd(x, max_lag=None, normalize=False) -> tuple[np.ndarray, np.ndarray]
```

Compute the empirical Mean Square Displacement of a 1D time series.

Formula: $\text{MSD}(\Delta) = (1 / (N - \Delta)) \cdot \sum_i [x(i + \Delta) - x(i)]^2$

Parameter	Type	Default	Description
<code>x</code>	array-like (1D)	<i>required</i>	Fluctuating observable. Accepts ndarray, pd.Series, list. Isolated NaN values are replaced by linear interpolation before computation.
<code>max_lag</code>	int or None	None	Maximum lag. Defaults to $N//2$. Hard-capped at $N//2$ (beyond $N/2$, each MSD estimate uses fewer than $N/2$ data pairs and becomes unreliable).
<code>normalize</code>	bool	False	If True, divide all MSD values by $\text{MSD}[0]$ (lag=1). Preserves the scaling exponent μ while setting the initial value to 1.

Returns: (lags, msd) — both shape (max_lag,). lags are integer steps 1...max_lag.

Raises: ValueError if input is not 1D (message: `X Input must be 1D. Got shape ...`); if fewer than 10 points (message: `X Need at least 10 data points. Got ...`); if more than 50% NaN (message: `X Too many missing values (...% NaN). Check your data.`).

```
time, values, meta = wn.read_csv('sunspot.csv')
lags, msd = wn.compute_msd(values)
print(lags.shape)  # (N//2,) e.g. (150,) for 300 points

# Inspect shape before fitting
wn.plot_msd_empirical(lags, msd, metadata=meta)
```

fit_msd()

```
fit_msd(lags, msd_empirical, model='cosine', p0=None, bounds=None,
        max_lag_fraction=1.0) -> FitResult | None
```

Fit $N \cdot \text{msd_theory}(T, *params)$ to empirical MSD using Levenberg–Marquardt (scipy.optimize.curve_fit). N is a normalization scalar always included alongside the physical parameters.

Parameter	Type	Default	Description
-----------	------	---------	-------------

lags	ndarray	<i>required</i>	Lag values from <code>compute_msd()</code> .
msd_empirical	ndarray	<i>required</i>	MSD values from <code>compute_msd()</code> .
model	str	'cosine'	SWNA model name.
p0	list or None	None	Initial guess for physical parameters only (N=1.0 appended internally). If None, uses built-in defaults per model.
bounds	tuple or None	None	Bounds as (lower_list, upper_list) for physical params only. N bounds [0.01, inf] appended automatically. If None, uses built-in defaults.
max_lag_fraction	float	1.0	Fraction of lags [0, 1] to use in fitting. E.g. 0.5 uses only the first half of lags.

Returns: `FitResult` , or `None` (never raises — prints `X` on failure).

On $R^2 < 0.5$ after a successful fit, prints `⚠ Low R² (...)`. Consider trying other models: ...

Default p0 and bounds per model (physical params only; N is appended as [0.01, ∞] always):

Model	Default p0	Lower bounds	Upper bounds
cosine	[1.2, 0.01]	[0.01, 1e-9]	[5.0, 10.0]
exponential	[1.2, 0.1]	[0.01, 1e-9]	[5.0, 100.0]
sine	[1.2, 0.01]	[0.01, 1e-9]	[5.0, 10.0]
fbm	[0.6]	[0.01]	[2.0]
exp_plateau	[5.0, 0.01, 3.0]	[0.01, 1e-9, 0.01]	[1000, 100, 1000]
inc_gamma	[1.5, 0.1]	[0.01, 1e-9]	[10.0, 100.0]
bessel_jmu_nu	[1.0, 0.5]	[0.01, -0.99]	[10.0, 10.0]

```
# Standard use (most common – done automatically inside analyze())
lags, msd = wn.compute_msd(values)
result = wn.fit_msd(lags, msd, model='exponential')

if result is not None:
    print(result.params['mu'])    # e.g. 1.15
    print(result.r_squared)      # e.g. 0.982

# Custom initial guess
result = wn.fit_msd(lags, msd, model='cosine',
                    p0=[2.0, 0.005])

# Use only first 60% of lags in fitting
```

```
result = wn.fit_msd(lags, msd, model='cosine',
                    max_lag_fraction=0.6)
```

FitResult (dataclass)

Returned by `fit_msd()`. All fields are always present when fit succeeds.

Field	Type	Description
<code>params</code>	dict	Fitted parameter values. Keys depend on model, e.g. <code>{'mu': 1.15, 'beta': 0.08, 'N': 2.3}</code> .
<code>std_errors</code>	dict	Standard errors <code>sqrt(diag(pcov))</code> , same keys as <code>params</code> .
<code>confidence_intervals</code>	dict	95% CIs as (lo, hi) tuples. CI = param $\pm$ 1.96 $\times$ SE, same keys as <code>params</code> .
<code>r_squared</code>	float	Best R^2 from the winning fit mode (pure or scaled).
<code>r_squared_pure</code>	float	R^2 when fitting <code>msd_theory(T, *params)</code> directly (N=1 fixed). NaN if pure fit failed.
<code>r_squared_scaled</code>	float	R^2 when fitting <code>N*msd_theory(T, *params)</code> with N free. NaN if scaled fit failed.
<code>fit_mode</code>	str	'pure' or 'scaled' — which fitting mode was selected (higher R^2 wins).
<code>model</code>	str	Name of the fitted model.
<code>lags_used</code>	ndarray	Lag values after <code>max_lag_fraction</code> truncation.
<code>msd_fitted</code>	ndarray	Theoretical curve evaluated at <code>lags_used</code> using the winning fit mode.

Dual-fit logic: `fit_msd()` always tries both fitting modes. *Pure* mode fits the theoretical MSD directly (N fixed to 1). *Scaled* mode fits N*MSD_theory (N free), which absorbs data amplitude. The mode with higher R^2 is selected and recorded in `fit_mode`. Both R^2 values are always reported in summaries so researchers can see which formulation fits better for their data.

.summary() — Returns a formatted box-drawing string showing all params, SEs, CIs, and R^2 :

```
print(result.fit.summary())
#
# | Fit Summary |
# | Model : cosine          R² = 0.9823 |
# |-----|
# | μ      = 1.2341 ± 0.0082 |
# |      95% CI: (1.218, 1.250) |
# | ν      = 0.0082 ± 0.0003 |
# |      95% CI: (0.008, 0.009) |
# | N      = 2.4312 ± 0.0441 |
```

#	95% CI: (2.345, 2.518)
#	

Parameter order in `FitResult.params` :

- **cosine, sine:** μ , ν , N
- **exponential:** μ , β , N
- **fbm:** H , N
- **exp_plateau (alias: dna):** a , b , c , N
- **inc_gamma:** ν , μ , N
- **bessel_jmu_nu:** μ , ν , N

9. Models

The MODELS registry contains 19 models (18 theoretical MSD formulas from the SWNA reference literature, plus the `exp_plateau` model from Violanda et al. 2019). Model selection is always *manual* — the researcher specifies `model=`. The package never auto-selects. Run `wn.list_models()` to see all models and their current status. The name `'dna'` is a backward-compatible alias for `'exp_plateau'`.

Available models (fully implemented)

Name	Row	Params	MSD Formula	Published application
<code>fbm</code>	1	H	T^{2H}	Benchmark; $H=0.5$ is Brownian
<code>sin_half</code>	2	—	$1 - \cos(T)$	No parameters; oscillatory
<code>cos_half</code>	3	—	$\sin(T)$	No parameters; oscillatory
<code>exponential</code>	4	μ, β	$\Gamma(\mu) \cdot \beta^{-\mu} \cdot T^{\mu-1} \cdot e^{-\beta/T}$	Earthquakes (Roque et al. 2024, $\mu \approx 1.00-1.19$); Sunspots (Toledo et al. 2024, $\mu \approx 1.15$)
<code>sine</code>	9	μ, ν	$\sqrt{\pi} \cdot \Gamma(\mu) \cdot \sin(\nu T/2) \cdot J_{\mu-1/2}(\nu T/2) \cdot (T/\nu)^{\mu-1/2}$	Planktonic predator–prey (Tuñaco et al., unpublished)
<code>cosine</code>	10	μ, ν	$\sqrt{\pi} \cdot \Gamma(\mu) \cdot \cos(\nu T/2) \cdot J_{\mu-1/2}(\nu T/2) \cdot (T/\nu)^{\mu-1/2}$	GBR coral (Elnar et al. 2021, $\mu \approx 4.64$); CO&sub2; (Elnar et al. 2024, $\mu \approx 0.91-0.97$)
<code>inc_gamma</code>	14	ν, μ	$\Gamma(\nu) \cdot P(\nu, \mu T) / \mu^\nu$	Restricted diffusion (plateau shape)
<code>bessel_j0_cos</code>	17	—	$J_0(T) - \cos(T)$	No parameters; Bessel convolution identity
<code>bessel_jmu_nu</code>	18	μ, ν	$J_{\mu+\nu}(T) / \mu$	Oscillatory Bessel model
<code>exp_plateau</code>	—	a, b, c	$a - c \cdot e^{-bL}$	Purely exponential memory, plateau MSD. First applied to DNA by Violanda et al. (2019, Phys. Scr. 94, 125006). Alias: <code>'dna'</code> .

Stub models (not yet implemented)

These are registered in the MODELS registry but raise `NotImplementedError` when called.

Name	Row	Params	Description
<code>exp_whittaker</code>	5	μ, β, ν	Power-law memory with Whittaker W function
<code>bessel_K</code>	6	μ, β	Power-law memory with modified Bessel K
<code>hypergeom_F1</code>	7	μ, β, ν	Power-law memory with Appell hypergeometric F1
<code>bessel_I</code>	8	μ, β	Power-law memory with modified Bessel I
<code>hypergeom_3F2</code>	11	μ, β, λ	Power-law with generalized hypergeometric ${}_3F_2$
<code>csc_power</code>	12	ν, c	Cosecant power-law modulation
<code>cot_power</code>	13	ν, c	Cotangent power-law modulation
<code>bessel_pair</code>	15	ν	Bessel function pair (no memory parameter)
<code>bessel_pair2</code>	16	ν, μ	Bessel function pair with memory parameter

Parameter reference

Symbol	Name	Models	Physical meaning
μ	mu	cosine, sine, exponential, <code>bessel_jmu_nu</code>	Memory parameter. $\mu=1$ is ordinary Brownian motion; $\mu<1$ indicates slower-than-linear MSD growth, $\mu>1$ indicates faster-than-linear MSD growth.
ν	nu	cosine, sine, <code>bessel_jmu_nu</code>	Characteristic frequency (rad per time unit). Controls oscillation period.
β	beta	exponential	Exponential decay rate (inverse time unit). Controls MSD shape at small lags.
H	H	fbm	Hurst exponent. $H=0.5$ is ordinary Brownian motion; $H<0.5$ MSD grows slower than T, $H>0.5$ MSD grows faster than T. Range: (0, 1].
a, b, c	—	<code>exp_plateau</code>	a = plateau MSD value; b = saturation rate; c = initial depression (a>c required). Restricted (saturating) diffusion. Model alias: <code>'dna'</code> .
N	N	all models	Normalization scalar. Always fitted alongside physical params. Absorbs data amplitude. Does not affect μ .

Model selection guidance

One model per run: `analyze()` fits exactly one model. Selecting the right model is the researcher's responsibility. If unsure, start with `model='exponential'` (most commonly used in published SWNA

studies). If your data shows clear oscillatory structure, try `model='cosine'` . Use `wn.batch_model_search('mydata.csv')` to run all available models and compare R^2 values side by side.

R^2 range	Interpretation	Action
≥ 0.8	Good fit	Report μ with 95% CI. Results are reliable.
0.5 – 0.8	Moderate fit	Caution. Report but note uncertainty. Consider another model.
< 0.5	Poor fit	Model is likely wrong. The package prints a Δ warning and suggests alternatives. Try <code>batch_model_search()</code> .

list_models() and get_model()

```
list_models() -> None
get_model(name: str) -> dict
```

```
wn.list_models()    # prints formatted table of all 19 models and their status

info = wn.get_model('cosine')
info['msd']          # callable: msd_cosine(T, mu, nu)
info['params']       # ['mu', 'nu']
info['status']       # 'available'
info['n_params']     # 2
```

`get_model()` raises `ValueError` if the name is unknown; raises `NotImplementedError` if the model status is `'not_implemented'` .

10. Plotting

The package provides two sets of plotting functions. **Exploratory plots** (`viz/explore.py`) are fast and annotated — designed for interactive inspection. **Publication plots** (`viz/publish.py`) apply strict `rcParams` styling (serif font, inward ticks, `dpi=150`) suitable for papers and reports.

Headless server note: `import whitenoise as wn` does *not* import `matplotlib` at package level. Viz functions import `matplotlib` lazily on first call. On headless servers (no display), set the backend before calling any viz function: `import matplotlib; matplotlib.use("Agg")`

Exploratory plots — viz/explore.py

All exploratory functions return a `matplotlib.figure.Figure` . All accept `ax=None` (create new figure) and `show=True` (call `plt.show()`).

plot_series()

```
plot_series(x, values, metadata=None, title='', ax=None, show=True) -> Figure
```

Plot raw sequential data directly from `read_csv()` . Use this for a quick visual inspection before `analyze()` . No analysis needed.

Parameter	Type	Default	Description
x	array-like (1D)	<i>required</i>	Independent variable (time, index, etc.).
values	array-like (1D)	<i>required</i>	Observable values.
metadata	dict or None	None	The metadata dict from <code>read_csv()</code> . When provided, axis labels are set automatically from <code>x_label</code> and <code>y_label</code> .
title	str	''	Custom plot title. If empty, uses <code>metadata['y_name']</code> if available.
ax	Axes or None	None	Axes to draw on. Creates new figure (10×3.5) if None.
show	bool	True	If True, call <code>plt.show()</code> after drawing.

```
time, values, meta = wn.read_csv('sunspot.csv')
fig = wn.plot_series(time, values, metadata=meta)

# After detrending
fluct = wn.detrend(values, method='moving_average', window=11)
fig = wn.plot_series(time, fluct, metadata=meta, title='Sunspot fluctuations')
```

plot_msd_empirical()

```
plot_msd_empirical(lags, msd, metadata=None, title='', ax=None, show=True) -> Figure
```

Plot empirical MSD scatter from `compute_msd()` . Use this to inspect the MSD shape before fitting — useful for deciding which model to try. figsize (7×4.5) when `ax=None`.

Parameter	Type	Default	Description
lags	array-like	<i>required</i>	Lag array from <code>compute_msd()</code> .
msd	array-like	<i>required</i>	MSD values from <code>compute_msd()</code> .
metadata	dict or None	None	From <code>read_csv()</code> . Provides lag unit for x-axis label automatically.
title	str	''	Custom plot title.
ax, show			Standard ax/show parameters.

```
lags, msd = wn.compute_msd(values)
fig = wn.plot_msd_empirical(lags, msd, metadata=meta)
```

plot_msd()

```
plot_msd(result, ax=None, show=True) -> Figure
```

Plot empirical MSD scatter and fitted theoretical MSD curve overlay, with R^2 in the legend label. Requires an `AnalysisResult` with a successful fit. `figsize (7×4.5)` when `ax=None`.

```
result = wn.analyze('sunspot.csv', model='exponential')
fig = wn.plot_msd(result)
```

plot_pdf()

```
plot_pdf(result, lag_index=None, ax=None, show=True) -> Figure
```

Plot empirical displacement histogram versus theoretical Gaussian PDF at a chosen lag. `figsize (6×4.5)` when `ax=None`.

Parameter	Type	Default	Description
<code>result</code>	<code>AnalysisResult</code>	<i>required</i>	Output of <code>analyze()</code> .
<code>lag_index</code>	int or None	<code>None</code>	Index into <code>result.lags</code> selecting the lag T at which PDF is evaluated. Defaults to <code>len(result.lags) // 10</code> .
<code>ax, show</code>			Standard parameters.

plot_timeseries()

```
plot_timeseries(result, ax=None, show=True) -> Figure
```

Plot `result.values` vs `result.time` — the preprocessed observable stored in the result. `figsize (9×3.5)` when `ax=None`.

plot_diagnostics()

```
plot_diagnostics(result, show=True) -> Figure
```

Two-by-two diagnostic figure: [time series | MSD] [PDF | parameter text box]. Returns Figure. `figsize (12×8)`.

```
fig = wn.plot_diagnostics(result, show=False)
fig.savefig('diagnostics.png', dpi=150, bbox_inches='tight')
```

Publication plots — viz/publish.py

This module has exactly **one** function in the public `wn.*` namespace: `publish_comparison()`. It applies publication rcParams (serif font, inward ticks, dpi=150) and is the only plotting function with a built-in `save_path=` parameter.

`wn.publish_msd`, `wn.publish_pdf`, and `wn.plot_all` **do not exist**. `publish_msd()` / `publish_pdf()` are defined in `viz/publish.py` but are thin, deprecated aliases — `publish_msd(result, **kwargs)` just calls `plot_msd(result, **kwargs)`, which only accepts `ax=` / `show=` (no `palette`, `figsize`, or `save_path`). They are also **not imported into** `whitenoise/__init__.py`, so `wn.publish_msd(...)` raises `AttributeError` regardless. There is also no `plot_all()` function anywhere in the package, and no `result.plot_all()` / `result.plot_msd()` / `result.plot_pdf()` shortcut methods on `AnalysisResult`.

For an MSD or PDF plot, use `wn.plot_msd(result)` / `wn.plot_pdf(result)` from Section 10 (styled with the default exploratory palette) and save the returned `Figure` yourself with `fig.savefig(...)`. For the full multi-panel view, use `wn.plot_diagnostics(result)`.

publish_comparison()

```
publish_comparison(cr, palette='default', figsize=None, save_path=None, show=True) -> Figure
```

Publication-quality μ comparison bar chart with 95% CI error bars. One horizontal bar per system, auto-sized figure if `figsize=None`.

Parameter	Type	Default	Description
<code>cr</code>	<code>ComparisonResult</code>	<i>required</i>	Output of <code>compare()</code> or <code>batch_analyze()</code> .
<code>palette</code>	<code>str</code>	<code>'default'</code>	<code>'default'</code> or <code>'colorblind'</code> .
<code>figsize</code>	<code>tuple</code> or <code>None</code>	<code>None</code>	Figure size. Defaults to <code>(7, max(3, 0.5*n_systems + 1.5))</code> if <code>None</code> .
<code>save_path</code>	<code>str</code> or <code>None</code>	<code>None</code>	If provided, saves the figure (format inferred from extension: pdf/png/svg/eps/jpg; unrecognized extensions default to pdf). Directory created automatically.
<code>show</code>	<code>bool</code>	<code>True</code>	If <code>True</code> , call <code>plt.show()</code> .

Raises: `ValueError` if `palette` is not `'default'` or `'colorblind'`.

```
comp = wn.compare({'A': 'a.csv', 'B': 'b.csv'}, model='cosine')
fig = wn.publish_comparison(comp, save_path='comparison.pdf')
```

11. Multi-Dataset Analysis

compare()

```
compare(datasets, model='cosine', detrend_method=None, normalize=False,
        max_lag_fraction=1.0, fit_kwargs=None) -> ComparisonResult
```

Analyze multiple datasets with the same model and collect results for comparison.

Parameter	Type	Default	Description
datasets	list or dict	<i>required</i>	list[str]: CSV file paths; labels auto-set from filenames (stem). dict: keys = labels (str), values = CSV paths (str) or 1D arrays.
model	str	'cosine'	SWNA model applied to all datasets.
detrend_method	str or None	None	Detrending method for all datasets.
normalize	bool	False	Whether to z-score normalize before fitting.
max_lag_fraction	float	1.0	Fraction of lags used in fitting.
fit_kwargs	dict or None	None	Extra kwargs for <code>fit_msd()</code> .

```
# Dict of CSV paths
comp = wn.compare({
    'Earthquakes': 'earthquake_ph.csv',
    'Sunspots':    'sunspot.csv',
    'Coral':       'coral_gbr.csv',
}, model='exponential')

# Dict mixing paths and arrays
comp = wn.compare({
    'System A': 'data_a.csv',
    'System B': my_array,
}, model='cosine')

# List of CSV paths (labels from filenames)
comp = wn.compare(['a.csv', 'b.csv', 'c.csv'], model='cosine')

print(comp.summary_df)
wn.publish_comparison(comp)
```

Each dataset is analyzed with `verbose=False` internally (progress lines are suppressed). `compare()` prints a one-line summary per dataset: `[i/n] label → R2=...`

print_comparison()

```
print_comparison(cr: ComparisonResult) -> None
```

Prints a formatted ASCII table from any `ComparisonResult` (output of `compare()`, `batch_analyze()`, or `batch_model_search()`). Dataset names longer than 16 characters are truncated with `...`. Returns `None` — this only prints, it does not return the table as a string.

```
wn.print_comparison(comp)
```

```
# +-----+-----+-----+-----+-----+-----+
# | Dataset      | Model   | mu/H/a | nu/beta | R2      | Regime      |
# +-----+-----+-----+-----+-----+-----+-----+
# | A            | cosine  | 0.9241  | 0.0000  | 0.9746  | subdiffusive |
# | B            | cosine  | 1.0682  | 0.0000  | 0.9649  | superdiffusive |
# +-----+-----+-----+-----+-----+-----+-----+
```

batch_analyze()

```
batch_analyze(source, model='cosine', pattern='*.csv', detrend_method=None,
               normalize=False, max_lag_fraction=1.0, fit_kwargs=None, n_jobs=1)
-> ComparisonResult
```

Batch SWNA on multiple CSV files. Three input modes:

Mode	source type	What happens
A — Folder	str (directory path, or str not ending in .csv)	Reads all files matching <code>pattern</code> (default <code>'*.csv'</code>) in the folder. Labels from filenames.
B — List of CSV paths	list[str]	Reads each CSV explicitly. Labels from filenames.
C — Multi-column CSV	str ending in .csv	Each non-time column is analyzed as a separate system via <code>read_csv_multi()</code> . Labels from <code>y_name</code> .

Parameter	Type	Default	Description
<code>source</code>	str or list[str]	<i>required</i>	Folder path, multi-column CSV path, or list of CSV file paths.
<code>model</code>	str	<code>'cosine'</code>	SWNA model applied to all datasets.
<code>pattern</code>	str	<code>'*.csv'</code>	Glob pattern for folder mode (Mode A only).
<code>detrend_method</code>	str or None	<code>None</code>	Detrending method for all datasets.
<code>normalize</code>	bool	<code>False</code>	Whether to z-score normalize before fitting.
<code>max_lag_fraction</code>	float	<code>1.0</code>	Fraction of lags used in fitting.

<code>fit_kwargs</code>	dict or None	None	Extra kwargs for <code>fit_msd()</code> .
<code>n_jobs</code>	int	1	Number of parallel worker threads. <code>n_jobs=1</code> is serial. <code>n_jobs>1</code> uses <code>ThreadPoolExecutor</code> . Parallel mode requires CSV file paths as input (not arrays).

```
# Mode A: folder
batch = wn.batch_analyze('data/xray_binaries/', model='cosine')

# Mode A: with glob pattern
batch = wn.batch_analyze('data/', model='exponential', pattern='*earthquake*.csv')

# Mode B: explicit list
batch = wn.batch_analyze(
    ['chile.csv', 'japan.csv', 'ph.csv'],
    model='exponential'
)

# Mode C: multi-column CSV
batch = wn.batch_analyze('climate.csv', model='cosine')
# Climate CSV has: time [yr], co2 [ppm], temperature [°C], sst [°C]
# Result has 3 systems: co2, temperature, sst

# Parallel (n_jobs > 1 requires CSV paths as input)
batch = wn.batch_analyze('data/', model='cosine', n_jobs=4)

print(batch.summary_df)
wn.export_summary(batch, 'batch_results.csv')
```

batch_model_search()

```
batch_model_search(path, models=None, detrend_method=None, normalize=False,
                    max_lag_fraction=1.0, fit_kwargs=None) -> ComparisonResult
```

Fit every available model to a single CSV file. Stub models are silently skipped. Returns a `ComparisonResult` sorted by R^2 descending.

Parameter	Type	Default	Description
<code>path</code>	str	<i>required</i>	Path to a whitenoise-format CSV file.
<code>models</code>	list[str] or None	None	Models to try. None → all available models from the MODELS registry.
<code>detrend_method</code>	str or None	None	Detrending method.
<code>normalize</code>	bool	False	Whether to z-score normalize before fitting.
<code>max_lag_fraction</code>	float	1.0	Fraction of lags used in fitting.

`fit_kwargs`

dict or None

None

Extra kwargs for `fit_ms()`.

```
search = wn.batch_model_search('sunspot.csv')
# Prints: cosine → R²=0.982, exponential → R²=0.961, fbm → R²=0.834, ...

print(search.summary_df[['model', 'mu', 'r_squared']].to_string())
# Best model is in the first row (sorted by R² descending)
best_model = search.summary_df.iloc[0]['model']
print(f'Best model: {best_model}')
```

Tip: Use `batch_model_search()` as your starting point when you are unfamiliar with which SWNA model applies to your data. Then re-run `analyze()` with the best model explicitly for the final result and plots.

ComparisonResult

`ComparisonResult` is a plain container — it has **no methods at all**, only three attributes:

Field	Type	Description
<code>results</code>	<code>list[AnalysisResult]</code>	Successfully analyzed results (failed runs are excluded from this list but appear as NaN rows in <code>summary_df</code>).
<code>models_used</code>	<code>list[str]</code>	Distinct model names used across all runs.
<code>summary_df</code>	<code>pd.DataFrame</code>	One row per dataset/model. Columns: <code>dataset_name</code> , <code>model</code> , <code>mu</code> , <code>mu_ci</code> , <code>nu_or_beta</code> , <code>nu_or_beta_ci</code> , <code>N</code> , <code>r_squared</code> , <code>r_squared_pure</code> , <code>r_squared_scaled</code> , <code>fit_mode</code> , <code>regime</code> .

No `.to_csv()` / `.to_excel()` methods. To save a `ComparisonResult`, use the standalone `export_summary()` function (Section 12) — `wn.export_csv()` does **not** accept a `ComparisonResult` (it only handles `AnalysisResult`) and will raise `AttributeError` if you pass one.

```
wn.export_summary(comparison, 'results.csv')    # correct
wn.export_csv(comparison, 'results.csv')        # WRONG – AttributeError
```

Note on `summary_df` columns: The `mu` column holds μ for cosine/sine/exponential, H for fbm, and a for the exp_plateau model. The `nu_or_beta` column holds ν for cosine/sine, β for exponential, and NaN for fbm, exp_plateau, and parameter-free models. The columns `r_squared_pure`, `r_squared_scaled`, and `fit_mode` reflect the dual-fit results (see Section 8). `regime` is the model-family classification label (see Section 1) and is `None` for models without a confirmed scheme.

12. Export

export_csv()

```
export_csv(result: AnalysisResult, path: str) -> None
```

Save an `AnalysisResult`'s MSD data to a 3-column CSV: `lag`, `msd_empirical`, `msd_fitted`. `msd_fitted` is the theoretical curve evaluated at each lag (rows outside the fitting window, or where the model returns a non-finite value, are written as `NaN`). Prints  Saved to {path}.

Takes only `AnalysisResult`, not `ComparisonResult`. There is no type dispatch inside `export_csv()` — passing a `ComparisonResult` raises `AttributeError` (it has no `.fit` / `.lags` / `.msd_empirical`). Use `export_summary()` below for `ComparisonResult` instead.

```
wn.export_csv(result, 'sunspot_msd.csv')
# lag,msd_empirical,msd_fitted
# 1.0,0.12345,0.11980
# 2.0,0.24100,0.23750
# ...
```

export_summary()

```
export_summary(cr: ComparisonResult, path: str) -> None
```

Save `ComparisonResult.summary_df` to CSV (no index column). This is the correct way to export a `compare()` / `batch_analyze()` / `batch_model_search()` result.

```
wn.export_summary(comparison, 'summary.csv')
```

export_series()

```
export_series(values, path, label='value', unit='', time=None,
              time_label='index', time_unit='step') -> None
```

Save a 1-D observable or preprocessed array to a whitenoise-format 2-column CSV ({time_label} [{time_unit}], {label} [{unit}]), reloadable directly with `read_csv()` to re-enter the pipeline without recomputing. Useful for checkpointing data before `compute_msd()` — e.g. after detrending, or after extracting genomic transition distances.

Parameter	Type	Default	Description
<code>values</code>	array-like (1D)	<i>required</i>	The observable values to save.
<code>path</code>	str	<i>required</i>	Destination file path.
<code>label</code>	str	'value'	Column name for the values.
<code>unit</code>	str	''	Unit string for the values column (empty = unitless).

<code>time</code>	array-like or None	None	Index/time axis. Defaults to <code>[0, 1, ..., N-1]</code> .
<code>time_label</code>	str	'index'	Column name for the time/index column.
<code>time_unit</code>	str	'step'	Unit string for the time/index column.

```
fluct = wn.detrend(values, method='moving_average', window=11)
wn.export_series(fluct, 'sunspot_fluct.csv', label='sunspot_fluctuation', unit='count')

# Reload later and skip straight to compute_msd:
_, fluct, _ = wn.read_csv('sunspot_fluct.csv')
lags, msd = wn.compute_msd(fluct)
```

Large arrays — recompute, don't save. For very large derived arrays (e.g. tens of millions of genomic nucleotide-separation distances), reloading a saved CSV can be slower than just recomputing from the source. Rule of thumb: if `len(values) > 1_000_000` , recompute instead of round-tripping through CSV. Use `export_msd()` below to checkpoint the (much smaller) MSD arrays instead.

export_msd()

```
export_msd(lags, msd, path, label='msd') -> None
```

Save `(lags, msd)` from `compute_msd()` to a 2-column `lag [step], {label} []` CSV, reloadable with `read_csv()` and passed directly to `fit_msd()` — lets you try different models without recomputing the MSD.

Parameter	Type	Default	Description
<code>lags</code>	array-like (1D)	<i>required</i>	Lag values from <code>compute_msd()</code> .
<code>msd</code>	array-like (1D)	<i>required</i>	Empirical MSD values from <code>compute_msd()</code> .
<code>path</code>	str	<i>required</i>	Destination file path.
<code>label</code>	str	'msd'	Column name for the MSD values.

```
lags, msd = wn.compute_msd(distances, max_lag=1500)
wn.export_msd(lags, msd, 'msd_A_A.csv')

# Reload later and fit a different model:
lags, msd, _ = wn.read_csv('msd_A_A.csv')
fit = wn.fit_msd(lags, msd, model='exponential')
```

Saving plots

None of the exploratory or publication plot functions have a `save_path` parameter (that parameter only exists on `publish_comparison()`) — save any returned `Figure` manually with `fig.savefig()`:

```
fig = wn.plot_diagnostics(result, show=False)
fig.savefig('diagnostics.png', dpi=150, bbox_inches='tight')

fig = wn.plot_msd(result, show=False)
fig.savefig('msd.pdf', bbox_inches='tight')

# publish_comparison() is the one exception – it has save_path= built in:
fig = wn.publish_comparison(comparison, save_path='comparison.pdf')
```

13. Advanced Usage

Running the pipeline step by step

For full control, call each step individually:

```
import whitenoise as wn
import numpy as np

# Step 1: Load
time, values, meta = wn.read_csv('sunspot.csv')

# Step 2: Inspect
fig = wn.plot_series(time, values, metadata=meta, show=False)

# Step 3: Detrend (manual decision)
fluct = wn.detrend(values, method='moving_average', window=11)

# Step 4: Compute MSD
lags, msd = wn.compute_msd(fluct)

# Step 5: Inspect MSD shape
wn.plot_msd_empirical(lags, msd, metadata=meta)

# Step 6: Fit with custom initial guess
fit = wn.fit_msd(lags, msd, model='exponential',
                 p0=[1.5, 0.05],
                 max_lag_fraction=0.8)

if fit is not None:
    print(fit.summary())
    print(f'mu = {fit.params[\"mu\"]:.4f}')
    lo, hi = fit.confidence_intervals['mu']
    print(f'95% CI: ({lo:.4f}, {hi:.4f})')
```

Custom po and bounds

If the default initial guesses fail (fit returns None or R^2 is low), supply `p0` and/or `bounds` based on domain knowledge. Remember: **N is always appended internally** — do not include it in `p0` or `bounds`.

```
# Custom p0 for cosine model (physical params mu, nu only; N appended internally)
result = wn.analyze('coral.csv', model='cosine',
                    fit_kwargs={'p0': [4.5, 0.003]})

# Custom p0 and bounds
result = wn.analyze('coral.csv', model='cosine',
                    fit_kwargs={
                        'p0': [4.5, 0.003],
                        'bounds': ([0.5, 1e-9], [10.0, 1.0]),
                    })
```

Accessing the model registry directly

```
from whitenoise.core.models import MODELS, msd_cosine, pdf_cosine
import numpy as np

T = np.linspace(1, 100, 200)
msd_vals = msd_cosine(T, mu=1.3, nu=0.008)

info = wn.get_model('exponential')
msd_fn = info['msd']
msd_vals = msd_fn(T, 1.15, 0.1)  # msd_exponential(T, mu, beta)
```

Headless server (no display)

```
import matplotlib
matplotlib.use('Agg')  # must be set BEFORE importing whitenoise viz functions
import whitenoise as wn

result = wn.analyze('sunspot.csv', model='exponential')
fig = wn.plot_diagnostics(result, show=False)
fig.savefig('output.png', dpi=150, bbox_inches='tight')
```

Building a summary DataFrame manually

`AnalysisResult` has no `.to_dict()` — pull the fields you want directly from `result.fit.params` / `.confidence_intervals`, or just use `wn.compare()` (Section 11), which builds `summary_df` for you.

```
import whitenoise as wn
import pandas as pd

systems = {
    'Earthquakes': 'earthquake_ph.csv',
    'Sunspots': 'sunspot.csv',
```

```

}
results = []
for label, path in systems.items():
    r = wn.analyze(path, model='exponential', verbose=False)
    row = {'label': label, 'mu': r.fit.params['mu'], 'r_squared': r.fit.r_squared}
    results.append(row)

df = pd.DataFrame(results)
print(df[['label', 'mu', 'r_squared']])

# Equivalent, built-in:
comp = wn.compare(systems, model='exponential')
print(comp.summary_df[['dataset_name', 'mu', 'r_squared']])

```

Methodological note: fitting method

Current fitting uses unweighted SSR (ordinary least squares), consistent with the published SWNA prior published results. A more statistically rigorous alternative would be **Weighted Least Squares (WLS)** with weights $w_{\Delta} = N - \Delta$ (large-lag MSD estimates are noisier, computed from fewer data pairs). WLS is not yet implemented but references for the approach include: Michalet (2010) and Vestergaard et al. (2014).

14. Full API Quick Reference

I/O

Function	Signature	Returns
<code>read_csv</code>	<code>read_csv(path)</code>	<code>tuple[ndarray, ndarray, dict]</code>
<code>read_csv_multi</code>	<code>read_csv_multi(path)</code>	<code>list[tuple[ndarray, ndarray, dict]]</code>
<code>export_csv</code>	<code>export_csv(result, path)</code>	None (AnalysisResult only)
<code>export_summary</code>	<code>export_summary(cr, path)</code>	None (ComparisonResult only)
<code>export_series</code>	<code>export_series(values, path, label='value', unit='', time=None, time_label='index', time_unit='step')</code>	None
<code>export_msdl</code>	<code>export_msdl(lags, msd, path, label='msd')</code>	None

Genomics (`wn.genomics`)

Function	Signature	Returns
<code>wn.genomics.read_fasta</code>	<code>read_fasta(path)</code>	<code>list[dict]</code>
<code>wn.genomics.read_fasta_single</code>	<code>read_fasta_single(path)</code>	<code>dict</code>
<code>wn.genomics.filter_sequence</code>	<code>filter_sequence(sequence, keep='ACGT')</code>	<code>str</code>
<code>wn.genomics.get_transition_distances</code>	<code>get_transition_distances(sequence, from_nuc, to_nuc)</code>	<code>ndarray[int64]</code>
<code>wn.genomics.get_all_transition_distances</code>	<code>get_all_transition_distances(sequence, nucleotides='ACGT')</code>	<code>dict[(str, str), ndarray]</code>
<code>wn.genomics.summarize_matrix</code>	<code>summarize_matrix(matrix)</code>	None (prints table)
<code>wn.genomics.extract_pair</code>	<code>extract_pair(seq, from_nuc, to_nuc, save_path=None)</code>	<code>ndarray</code>
<code>wn.genomics.compute_pair_msdl</code>	<code>compute_pair_msdl(source, max_lag=1500, save_path=None)</code>	<code>(lags, msd)</code>
<code>wn.genomics.fit_pair</code>	<code>fit_pair(source, model='exp_plateau', p0=None, ...)</code>	<code>FitResult</code>

<code>wn.genomics.plot_pair</code>	<code>plot_pair(lags, msd, fit=None, title='', save_path=None)</code>	Figure
<code>wn.genomics.analyze_pair</code>	<code>analyze_pair(source, from_nuc, to_nuc, model='exp_plateau', ...)</code>	dict
<code>wn.genomics.refit_pair</code>	<code>refit_pair(msd_csv_path, model='exp_plateau', p0=None, ...)</code>	(FitResult, Figure)

RadioJOVE (`wn.radiojove`)

Function	Signature	Returns
<code>wn.radiojove.list_burst_csvs</code>	<code>list_burst_csvs(folder, pattern='*.csv')</code>	list[str]
<code>wn.radiojove.parse_filename_metadata</code>	<code>parse_filename_metadata(filename)</code>	dict
<code>wn.radiojove.read_radiojove_csv</code>	<code>read_radiojove_csv(path)</code>	(time, intensity, metadata)
<code>wn.radiojove.group_by_cadence</code>	<code>group_by_cadence(paths)</code>	dict[float, list[str]]
<code>wn.radiojove.zscore_normalize</code>	<code>zscore_normalize(intensity)</code>	ndarray
<code>wn.radiojove.resample</code>	<code>resample(time, intensity, target_cadence)</code>	(time_new, intensity_new)
<code>wn.radiojove.analyze_burst</code>	<code>analyze_burst(path, model='exponential', detrend_method='linear', ...)</code>	dict
<code>wn.radiojove.plot_burst_msd</code>	<code>plot_burst_msd(result, title='', save_path=None)</code>	Figure
<code>wn.radiojove.batch_analyze_bursts</code>	<code>batch_analyze_bursts(source, model='exponential', ...)</code>	pd.DataFrame
<code>wn.radiojove.compare_bursts</code>	<code>compare_bursts(paths, model='exponential', ...)</code>	ComparisonResult

Preprocessing (optional)

Function	Signature	Returns
<code>detrend</code>	<code>detrend(values, method='linear', poly_order=1, window=7)</code>	ndarray
<code>detrend_psd</code>	<code>detrend_psd(values, threshold=0.5)</code>	ndarray
<code>detrend_fingerprint</code>	<code>detrend_fingerprint(values, cycle_length=None, trough_indices=None)</code>	ndarray

<code>normalize</code>	<code>normalize(values, method='zscore')</code>	<code>ndarray</code>
<code>smooth</code>	<code>smooth(values, window=5, method='moving_average')</code>	<code>ndarray</code>

Core

Function	Signature	Returns
<code>compute_msd</code>	<code>compute_msd(x, max_lag=None, normalize=False)</code>	<code>tuple[ndarray, ndarray]</code>
<code>fit_msd</code>	<code>fit_msd(lags, msd_empirical, model='cosine', p0=None, bounds=None, max_lag_fraction=1.0)</code>	<code>FitResult</code> <code>None</code>
<code>list_models</code>	<code>list_models()</code>	<code>None</code> (prints table)
<code>get_model</code>	<code>get_model(name)</code>	<code>dict</code>
<code>FitResult</code>	<code>dataclass</code>	<code>params, std_errors, confidence_intervals, r_squared, r_squared_pure, r_squared_scaled, fit_mode, model, lags_used, msd_fitted; method .summary() -> str</code>

Analysis

Function	Signature	Returns
<code>analyze</code>	<code>analyze(source, model='cosine', label='', time=None, detrend_method=None, normalize=False, max_lag_fraction=1.0, fit_kwargs=None, verbose=True)</code>	<code>AnalysisResult</code>
<code>AnalysisResult</code>	<code>dataclass</code>	<code>dataset_name, model, fit, lags, msd_empirical, values, time, metadata; properties .alpha, .memory_exponent, .regime (all model-family-specific, see Section 1); method .summary() -> None (prints). No .mu, .to_dict(), .to_csv(), or plot shortcuts.</code>
<code>compare</code>	<code>compare(datasets, model='cosine', detrend_method=None, normalize=False, max_lag_fraction=1.0, fit_kwargs=None)</code>	<code>ComparisonResult</code>
<code>print_comparison</code>	<code>print_comparison(cr)</code>	<code>None</code> (prints ASCII table)

<code>batch_analyze</code>	<code>batch_analyze(source, model='cosine', pattern='*.csv', detrend_method=None, normalize=False, max_lag_fraction=1.0, fit_kwargs=None, n_jobs=1)</code>	<code>ComparisonResult</code>
<code>batch_model_search</code>	<code>batch_model_search(path, models=None, detrend_method=None, normalize=False, max_lag_fraction=1.0, fit_kwargs=None)</code>	<code>ComparisonResult</code>
<code>ComparisonResult</code>	class	results, models_used, summary_df. No methods — use <code>export_summary()</code> to save.

Exploratory Plots (viz/explore.py)

Function	Signature	Returns
<code>plot_series</code>	<code>plot_series(x, values, metadata=None, title='', ax=None, show=True)</code>	Figure
<code>plot_msdpempirical</code>	<code>plot_msdpempirical(lags, msdp, metadata=None, title='', ax=None, show=True)</code>	Figure
<code>plot_msdp</code>	<code>plot_msdp(result, ax=None, show=True)</code>	Figure
<code>plot_pdf</code>	<code>plot_pdf(result, lag_index=None, ax=None, show=True)</code>	Figure
<code>plot_timeseries</code>	<code>plot_timeseries(result, ax=None, show=True)</code>	Figure
<code>plot_diagnostics</code>	<code>plot_diagnostics(result, show=True)</code>	Figure

Publication Plots (viz/publish.py)

Function	Signature	Returns
<code>publish_comparison</code>	<code>publish_comparison(cr, palette='default', figsize=None, save_path=None, show=True)</code>	Figure

This is the only function this module exports to `wn.*`. There is no `plot_all()` anywhere in the package. `publish_msdp` / `publish_pdf` exist in the source but are unexported deprecated aliases with no extra parameters — use `plot_msdp()` / `plot_pdf()` from the Exploratory Plots table above instead.

15. Genomics Extension (`wn.genomics`)

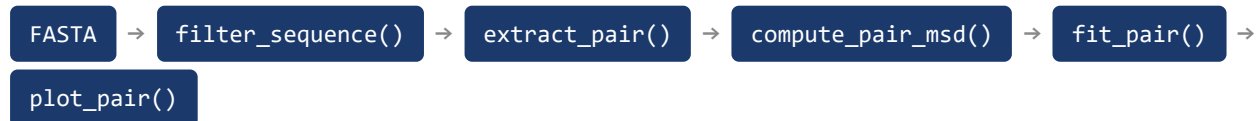
The `wn.genomics` submodule applies SWNA to genomic DNA sequences using the nucleotide separation distance approach from Violanda et al. (2019). The stochastic variable is the *distance between*

consecutive nucleotide occurrences along the sequence. This distance sequence feeds into the standard SWNA pipeline via the `exp_plateau` model.

No additional dependencies required. Uses only NumPy and the Python standard library. BioPython is optional — a convenience adapter is provided.

Full documentation: See whitenoise/genomics/GENOMICS_GUIDE.html for the complete step-by-step guide with all options. This section covers the essential API.

Pipeline overview



Or use `analyze_pair()` for the full pipeline in one call. Saved MSD CSVs can be reloaded with `refit_pair()` to retry fitting without recomputing.

15.1 • Loading FASTA

```
record = wn.genomics.read_fasta_single('chromosome1.fasta')
seq_raw = record['sequence']
print(f"Length: {len(seq_raw):,} bp, ambiguous: {record['n_ambiguous']: ,}")

# Remove ambiguous bases (W, N, R, Y, etc.)
seq = wn.genomics.filter_sequence(seq_raw)
```

15.2 • Transition distances

For a pair (from_nuc → to_nuc), distances are computed along the sequence:

- **Same-type** (A→A, G→G, ...): consecutive occurrences of the same nucleotide. Reproduces Violanda et al. (2019) exactly.
- **Cross-type** (A→C, ...): from each occurrence of from_nuc, distance to the nearest downstream to_nuc. Extension not in the original paper.

```
# Single pair
dist = wn.genomics.get_transition_distances(seq, 'A', 'A')

# All 16 pairs at once (more efficient than calling 16 times)
matrix = wn.genomics.get_all_transition_distances(seq)
wn.genomics.summarize_matrix(matrix) # prints count table
dist_AA = matrix[('A', 'A')]
```

Model choice for off-diagonal pairs: The `exp_plateau` model was derived for same-type (diagonal) distances. Off-diagonal pairs (A→C, A→G, ...) often yield low R^2 with `exp_plateau`. Try `model='exponential'` or `model='cosine'` for cross-type pairs.

15.3 · Step-by-step pipeline

```
import whitenoise as wn, os

record = wn.genomics.read_fasta_single('chromosome1.fasta')
seq = wn.genomics.filter_sequence(record['sequence'])

# Step 1: Extract distances (save for later)
dist = wn.genomics.extract_pair(seq, 'A', 'A', save_path='AA_distances.csv')

# Step 2: Compute MSD (save to avoid recomputing)
lags, msd = wn.genomics.compute_pair_msd(
    dist, max_lag=1500, save_path='AA_msd.csv'
)

# Step 3: Fit
fit = wn.genomics.fit_pair((lags, msd), model='exp_plateau')
print(f"R² = {fit.r_squared:.4f}")

# Step 4: Plot
fig = wn.genomics.plot_pair(lags, msd, fit,
                             title='A→A | exp_plateau model',
                             save_path='AA_fit.png')
```

Choosing max_lag: The exp_plateau model saturates at $L \approx 3/b$. With bacterial reference $b \approx 0.0024$, saturation is near $L = 1250$. `max_lag=1500` is a safe default that captures the full curve.

15.4 · Full single-pair pipeline (one call)

```
result = wn.genomics.analyze_pair(
    source='chromosome1.fasta',
    from_nuc='A', to_nuc='A',
    model='exp_plateau',
    max_lag=1500,
    save_dir='results/',      # saves distances CSV, MSD CSV, and plot PNG
)

print(result['fit'].r_squared)
result['figure'].show()
```

15.5 · Re-fitting from a saved MSD CSV

```
# Retry with a better initial guess
fit, fig = wn.genomics.refit_pair(
    'results/A_A_msd.csv',
    model='exp_plateau',
    p0=[27.0, 0.025, 2.0],  # estimates: a=plateau, b=6/L_plateau, c=a-MSD(1)
    save_path='results/A_A_refit.png',
)
```

```
# Try a different model for an off-diagonal pair
fit, fig = wn.genomics.refit_pair('results/A_C_msd.csv', model='exponential')

# Fit only first 70% of lags (exclude noisy plateau)
fit, fig = wn.genomics.refit_pair('results/A_A_msd.csv', max_lag_fraction=0.7)
```

15.6 • The exp_plateau model

$$\text{MSD}(L) = a - c \cdot \exp(-b \cdot L)$$

Parameter	Role	Typical range (bacterial)
a	Plateau height — MSD approaches a as $L \rightarrow \infty$	3–8
b	Exponential decay rate — how fast MSD rises to plateau	0.001–0.01
c	Amplitude of exponential term; must satisfy $c < a$	2–6
N	Normalization scalar — fitted automatically	~1

Published reference values (Violanda et al. 2019, bacterial genomes): $a \approx 5.21$, $b \approx 0.0024$, $c \approx 3.81$

No μ parameter. The exp_plateau model describes restricted diffusion (bounded MSD plateau) via the a, b, c parameters instead. The backward-compatible alias `model='dna'` also works.

15.7 • Batch analysis (all 16 pairs)

```
import whitenoise as wn, os

record = wn.genomics.read_fasta_single('chromosome1.fasta')
seq = wn.genomics.filter_sequence(record['sequence'])
os.makedirs('results', exist_ok=True)

for nuc in ['A', 'C', 'G', 'T']:
    dist = wn.genomics.extract_pair(seq, nuc, nuc)
    lags, msd = wn.genomics.compute_pair_msd(
        dist, max_lag=1500, save_path=f'results/{nuc}_{nuc}_msd.csv'
    )
    fit = wn.genomics.fit_pair((lags, msd), model='exp_plateau')
    wn.genomics.plot_pair(lags, msd, fit,
        title=f'{nuc}→{nuc}',
        save_path=f'results/{nuc}_{nuc}_fit.png')
```

15.8 • Genomics API reference

Function	Location	Description
----------	----------	-------------

<code>read_fasta(path)</code>	<code>wn.genomics</code>	Load all records from a FASTA file → list[dict]
<code>read_fasta_single(path)</code>	<code>wn.genomics</code>	Load first record only → dict
<code>filter_sequence(seq)</code>	<code>wn.genomics</code>	Remove non-ACGT characters → str
<code>get_transition_distances(seq, from, to)</code>	<code>wn.genomics</code>	Single pair distances → ndarray
<code>get_all_transition_distances(seq)</code>	<code>wn.genomics</code>	Full 4×4 distance matrix → dict
<code>summarize_matrix(matrix)</code>	<code>wn.genomics</code>	Print count table
<code>extract_pair(seq, from, to, save_path)</code>	<code>wn.genomics</code>	Extract distances + optionally save CSV
<code>compute_pair_msd(source, max_lag, save_path)</code>	<code>wn.genomics</code>	MSD from array or CSV; optional save
<code>fit_pair(source, model, p0, ...)</code>	<code>wn.genomics</code>	Fit from (lags, msd) tuple or MSD CSV → FitResult
<code>plot_pair(lags, msd, fit, ...)</code>	<code>wn.genomics</code>	Single-pair MSD plot → Figure
<code>analyze_pair(source, from, to, ...)</code>	<code>wn.genomics</code>	Full single-pair pipeline → dict
<code>refit_pair(msd_csv, model, p0, ...)</code>	<code>wn.genomics</code>	Reload saved MSD and refit → (FitResult, Figure)

Short sequences (COI barcodes, ~658 bp). Each distance array contains only ~50–150 values per pair. MSD plots will be noisy and the plateau may be hard to resolve. Interpret results with caution and consider pooling sequences across samples.

16. RadioJOVE Extension (`wn.radiojove`)

The `wn.radiojove` submodule applies SWNA to radio burst observations recorded with a RadioJOVE spectrograph. The stochastic variable is the Z-score normalized intensity of a selected burst region sampled at a fixed cadence (0.1 s or 1.0 s). The memory parameter μ quantifies the dynamical memory structure of the burst.

Full documentation: See [whitenoise/radiojove/RADIOJOVE_GUIDE.html](https://whitenoise.github.io/radiojove/RADIOJOVE_GUIDE.html) for the complete guide including the preprocessing GUI workflow, multi-resolution comparison, and troubleshooting. This section covers the essential API.

Pipeline overview

Stage 1 — Preprocessing (manual GUI):

JPG spectrograph → region selection GUI → Z-score export → CSV file

Stage 2 — SWNA analysis (this package):

read_radiojove_csv() → detrend() (optional) → analyze_burst() → compare_bursts()

16.1 • CSV format and filename convention

The GUI exports CSVs with two columns and filenames following this pattern:

```
YYMMDDHHMMSS<Station>_region_<N>_<cadence>s.csv
```

Example:

```
230728154000Higgins_Home_region_1_0.1s.csv ← 0.1 s cadence
230728154000Higgins_Home_region_1_1.0s.csv ← 1.0 s cadence
```

Column 1: time in seconds. Column 2: Z-score normalized intensity (mean=0, std=1).

Why two cadences? Running SWNA on both 0.1 s and 1.0 s files and comparing μ values confirms that the memory structure is robust to temporal resolution. Use 1.0 s as the primary result and 0.1 s as validation. See Section 16.5.

16.2 • Reading and organizing data

```
import whitenoise as wn

# Read a single CSV
time, intensity, meta = wn.radiojove.read_radiojove_csv(
    '230728154000Higgins_Home_region_1_0.1s.csv'
)
print(f"N points : {len(time)}")
print(f"Cadence   : {meta['cadence_s']} s")
print(f"Station    : {meta['station']}")

# List all CSVs in a folder
paths = wn.radiojove.list_burst_csvs('Solar/Type 3 Bursts/first trials/')

# Group by cadence
groups = wn.radiojove.group_by_cadence(paths)
paths_01s = groups[0.1]  # 0.1-second cadence files
paths_10s = groups[1.0]  # 1.0-second cadence files
```

16.3 • Single burst analysis

```
out = wn.radiojove.analyze_burst(
    '230728154000Higgins_Home_region_1_0.1s.csv',
    model='exponential',
    detrend_method='linear',  # recommended default
    output_dir='swna_results/', # saves plot PNG here
)

result = out['result']
if result and result.fit:
```

```

print(f"mu      = {result.fit.params['mu']:.4f}")
print(f"R2      = {result.fit.r_squared:.4f}")

out['figure'].show()    # dual-panel MSD plot (linear + log-log)

```

Model choice: Use `model='exponential'` for Type III solar bursts and most Jovian S-bursts — these have monotonically growing MSDs. The cosine model fails catastrophically ($R^2 \approx -10^{60}$) on monotonic MSDs.

16.4 • Batch analysis

```

df = wn.radiojove.batch_analyze_bursts(
    'Solar/Type 3 Bursts/first trials/',
    model='exponential',
    output_dir='swna_results/',
)

print(df[['dataset', 'mu', 'mu_ci_low', 'mu_ci_high', 'r_squared']])

```

You can also pass a specific list of files:

```
df = wn.radiojove.batch_analyze_bursts(paths_01s, model='exponential')
```

Column	Description
dataset	Filename stem
mu	Memory parameter μ
mu_ci_low / mu_ci_high	95% CI bounds for μ
r_squared	Best R^2 (winning fit mode)
r_squared_pure	R^2 from pure MSD fit ($N=1$)
r_squared_scaled	R^2 from N -MSD fit
fit_mode	'pure' or 'scaled'

16.5 • Multi-resolution comparison (0.1 s vs 1.0 s)

```

import pandas as pd

groups = wn.radiojove.group_by_cadence(paths)

df_01s = wn.radiojove.batch_analyze_bursts(groups[0.1], model='exponential', verbose=False)
df_10s = wn.radiojove.batch_analyze_bursts(groups[1.0], model='exponential', verbose=False)

df_01s['cadence'] = '0.1s'
df_10s['cadence'] = '1.0s'

```



```
df_all = pd.concat([df_01s, df_10s], ignore_index=True)
print(df_all[['dataset', 'cadence', 'mu', 'r_squared']])
```

If μ at 0.1 s and 1.0 s agree within their 95% confidence intervals, the memory structure is robust. A discrepancy suggests resolution-dependent effects.

16.6 • Comparing bursts

```
cr = wn.radiojove.compare_bursts(groups[0.1], model='exponential')

wn.print_comparison(cr)          # ASCII table
wn.publish_comparison(cr)        # publication-quality mu bar chart

cr.summary_df.to_csv('comparison.csv', index=False)
```

16.7 • The exponential SWNA model for radiobursts

$$\text{MSD}(T, \mu, \beta) = \Gamma(\mu) * \beta^{(-\mu)} * T^{(\mu-1)} * \exp(-\beta / T)$$

Parameter	Role	Typical range (Type III bursts)
μ	Memory exponent — controls MSD growth rate	1.4–1.7
β	Exponential decay rate	10–100
N	Normalization scalar	~1

Type III solar bursts are driven by semi-relativistic electron beams spiralling along open coronal magnetic field lines, producing positively correlated intensity fluctuations consistent with $\mu \approx 1.4\text{--}1.6$ (**superdiffusive**).

16.8 • RadioJOVE API reference

Function	Location	Description
<code>list_burst_csvs(folder)</code>	<code>wn.radiojove</code>	List all CSVs in a folder, sorted
<code>parse_filename_metadata(filename)</code>	<code>wn.radiojove</code>	Extract datetime, station, region, cadence from filename
<code>read_radiojove_csv(path)</code>	<code>wn.radiojove</code>	Read CSV; returns (time, intensity, enriched metadata)
<code>group_by_cadence(paths)</code>	<code>wn.radiojove</code>	Group file list by time cadence → dict[float, list[str]]
<code>zscore_normalize(intensity)</code>	<code>wn.radiojove</code>	Z-score normalize (only if not already done by GUI)

<code>resample(time, intensity, cadence)</code>	<code>wn.radiojove</code>	Resample to a different cadence via linear interpolation
<code>analyze_burst(path, model, ...)</code>	<code>wn.radiojove</code>	Full SWNA pipeline for one burst CSV → dict
<code>plot_burst_msd(result, ...)</code>	<code>wn.radiojove</code>	Dual-panel MSD plot (linear + log-log) → Figure
<code>batch_analyze_bursts(source, ...)</code>	<code>wn.radiojove</code>	Batch analysis; returns summary DataFrame
<code>compare_bursts(paths, model, ...)</code>	<code>wn.radiojove</code>	Multi-burst comparison via <code>wn.compare()</code> → <code>ComparisonResult</code>

17. Validated Benchmarks

The following published results serve as benchmarks for validating your analysis setup.

System	Model	μ (published)	Reference
Philippine earthquakes (inter-event times)	exponential	$\mu \approx 1.00\text{--}1.19$	Roque et al. (2024)
Solar sunspot number	exponential	$\mu \approx 1.15$	Toledo et al. (2024)
Great Barrier Reef coral bleaching	cosine	$\mu \approx 4.64$	Elnar et al. (2021)
CO ₂ ; Keeling curve (Mauna Loa)	cosine	$\mu \approx 0.91\text{--}0.97$	Elnar et al. (2024)
DNA nucleotide separation (bacterial genomes)	exp_plateau	$a \approx 5.21, b \approx 0.0024, c \approx 3.81$	Violanda et al. (2019)

18. Known Limitations

1. Stub models. Nine models in the MODELS registry raise `NotImplementedError` when called: `exp_whittaker`, `bessel_K`, `hypergeom_F1`, `bessel_I`, `hypergeom_3F2`, `csc_power`, `cot_power`, `bessel_pair`, `bessel_pair2`. Use `wn.list_models()` to see current status.

2. Unweighted fitting. The current fitting uses ordinary least squares (unweighted SSR). Statistically, WLS with weights $w_\Delta = N - \Delta$ would be more rigorous (large-lag MSD estimates are noisier). Not implemented, but kept consistent with prior published results from the Bernido group.

3. Parallel `n_jobs > 1` requires CSV paths. When `n_jobs > 1` is specified in `batch_analyze()`, all jobs must use CSV file paths. In-memory arrays cannot be reliably passed to parallel worker threads.

4. Oscillatory model NaN regions. The cosine and sine models return `nan` where $\cos(\sqrt{T}/2) \leq 0$ or $\sin(\sqrt{T}/2) \leq 0$ respectively (the formula produces a non-positive MSD there, which has no physical meaning). If your MSD plot shows gaps, this is expected. Consider using a smaller lag range with `max_lag_fraction`, or try a different model.

5. Minimum data length. `compute_msd()` requires at least 10 data points. Fewer than ~50 points will produce very noisy MSD estimates and unreliable fits. Published results typically use $N \geq 200$ points.

19. Citation

If you use this package in published work, please cite:

White noise analysis: Some applications in complex systems, biophysics and quantum mechanics.
International Journal of Modern Physics B, 26, 1230014.
<https://doi.org/10.1142/S0217979212300149>

White noise functional integral for exponentially decaying memory:
nucleotide distribution in bacterial genomes.
Physica Scripta, 94, 125006.
<https://doi.org/10.1088/1402-4896/ab3342>

For specific models or applications, also cite the primary application papers listed in Section 17 (Validated Benchmarks).